**St. Francis Institute of Technology, Mumbai-400 103**
**Department Of Information Technology**

**A.Y. 2025-2026**
**Class: BE-ITA/B, Semester: VIII**
Subject: BlockChain Lab

# Experiment – 6

**Name: Durva Kadam** **Roll No: 23**

1. **Aim:** Case study on Hyperledger blockchain.
2. **Lab outcome:** After performing the experiment, the students will be able to **comprehend** the working of Hyperledger blockchain. (PO1, PSO1, BL2)

3. **Prerequisite:**
   Fundamental knowledge of blockchain
4. **Requirements:** The following are the requirements –
   PC and Internet.

5. **Pre-Experiment Theory:**
   What is Hyperledger?

Hyperledger is an open-source project created to support the development of blockchain-based distributed ledgers. Hyperledger consists of a collaborative effort to create the needed frameworks, standards, tools and libraries to build blockchains and related applications. Hyperledger was created by the Linux Foundation in 2016. Hyperledger Fabric, an open- source project from the Linux Foundation, is the modular blockchain framework and standard for enterprise blockchain platforms. Intended as a foundation for developing enterprise-grade applications and industry solutions, the open, modular architecture uses plug–and-play components to accommodate a wide range of use cases.

Hyperledger Composer is a set of collaboration tools for building blockchain business networks that make it simple and fast for business owners and developers to create smart contracts and blockchain applications to solve business problems. Built with JavaScript, leveraging modern tools including node.js, npm, CLI and popular editors, Composer offers business centric abstractions as well as sample apps with easy to test devops processes to create robust blockchain solutions that drive alignment across business requirements with technical development.

A smart contract in Hyperledger Fabric is a program, called chaincode which is written in Go, JavaScript, and other programming languages such as Java that implement a prescribed interface.

6. **Laboratory Exercise**
7. **Steps to be implemented.**
   Select any one research paper regarding case study on Hyperledger Blockchain, study it and write report based on your studies.
8. **Program Code**
   Write any one sample chaincode in Golang for hyperledger
9. **Post Experimental Exercise-**
10. **Questions:**
    How a chaincode is implemented on Hyperledger blockchain?
11. **Results/Observations/Program output:Present the program input/output results**

**11. CONCLUSION:**

a. Write what was performed in the experiment.
b. Write which tools you used to perform the experiment
c. Write what you inferred from the output obtained.

**12. References:**

[1] https://www.hyperledger.org/learn/case-studies
[2] https://hyperledger-fabric.readthedocs.io/en/release-2.2/whatis.h
[3] Hyperledger Fabric In-Depth: Learn, Build and Deploy Blockchain applications Using Hyperledger Fabric, Ashwani Kumar, BPB publications

***PAPER:***
***Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains***

- Authors: Elli Androulaki et al.
- Organization: IBM
- Conference: EuroSys 2018
- DOI: https://doi.org/10.1145/3190508.3190538
- Type: Peer-reviewed ACM conference paper

## Case Study Report

The research paper titled **"Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains"** was published in EuroSys in 2018. The main author of the paper is Elli Androulaki along with other researchers from IBM. The paper explains the architecture and working of Hyperledger Fabric, which is a permissioned blockchain platform developed under the Linux Foundation.

The objective of the paper is to overcome the limitations of traditional blockchain systems. Most existing blockchain platforms follow the Order-Execute model, which reduces system performance and requires deterministic smart contracts. This model also limits scalability and flexibility. To solve these issues, the authors propose a new architecture called Execute-Order-Validate, which separates transaction execution from ordering and validation.

**Code-**

```
package main
import (
	"encoding/json"
	"fmt"
	"github.com/hyperledger/fabric-contract-api-go/contractapi"
)
SmartContract provides functions for managing insurance policies
type SmartContract struct {
	contractapi.Contract
}
 Policy structure
type Policy struct {
	PolicyID string `json:"policyID"`
	Holder   string `json:"holder"`
	Type     string `json:"type"`
	Premium  int    `json:"premium"`
	Status   string `json:"status"`
}
CreatePolicy adds a new policy to the ledger
func (s *SmartContract) CreatePolicy(ctx contractapi.TransactionContextInterface, policyID string, holder string, policyType string,
premium int) error {
	exists, err := s.PolicyExists(ctx, policyID)
	if err != nil {
		return err
	}
	if exists {
		return fmt.Errorf("policy %s already exists", policyID)
	}
	policy := Policy{
		PolicyID: policyID,
		Holder:   holder,
		Type:     policyType,
		Premium:  premium,
		Status:   "ACTIVE",
	}
	policyJSON, err := json.Marshal(policy)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(policyID, policyJSON)
}
// QueryPolicy retrieves a policy from the ledger
```

```
func (s *SmartContract) QueryPolicy(ctx contractapi.TransactionContextInterface, policyID string) (*Policy, error) {

	policyJSON, err := ctx.GetStub().GetState(policyID)
	if err != nil {
		return nil, fmt.Errorf("failed to read from ledger: %v", err)
	}
	if policyJSON == nil {
		return nil, fmt.Errorf("policy %s does not exist", policyID)
	}

	var policy Policy
	err = json.Unmarshal(policyJSON, &policy)
	if err != nil {
		return nil, err
	}
	return &policy, nil
}
// UpdatePolicyStatus updates the status of a policy
func (s *SmartContract) UpdatePolicyStatus(ctx contractapi.TransactionContextInterface, policyID string, newStatus string) error {

	policy, err := s.QueryPolicy(ctx, policyID)
	if err != nil {
		return err
	}
	policy.Status = newStatus
	policyJSON, err := json.Marshal(policy)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(policyID, policyJSON)
}
// PolicyExists checks if a policy exists in ledger
func (s *SmartContract) PolicyExists(ctx contractapi.TransactionContextInterface, policyID string) (bool, error) {
	policyJSON, err := ctx.GetStub().GetState(policyID)
	if err != nil {
		return false, err
	}
	return policyJSON != nil, nil
}
func main() {
	chaincode, err := contractapi.NewChaincode(&SmartContract{})
	if err != nil {
		fmt.Printf("Error creating chaincode: %v", err)
package main
import (
	"encoding/json"
	"fmt"
	"github.com/hyperledger/fabric-contract-api-go/contractapi"
)
// SmartContract provides functions for managing insurance policies
type SmartContract struct {
	contractapi.Contract
}
// Policy structure
type Policy struct {
	PolicyID string `json:"policyID"`
	Holder   string `json:"holder"`
	Type     string `json:"type"`
	Premium  int    `json:"premium"`
	Status   string `json:"status"`
}
// CreatePolicy adds a new policy to the ledger
func (s *SmartContract) CreatePolicy(ctx contractapi.TransactionContextInterface, policyID string, holder string, policyType string,
premium int) error {
	exists, err := s.PolicyExists(ctx, policyID)
	if err != nil {
		return err
	}
	if exists {
		return fmt.Errorf("policy %s already exists", policyID)
	}
	policy := Policy{
		PolicyID: policyID,
		Holder:   holder,
		Type:     policyType,
		Premium:  premium,
		Status:   "ACTIVE",
```

```
	}
	policyJSON, err := json.Marshal(policy)
	if err != nil {
		return err
	}
	return ctx.GetStub().PutState(policyID, policyJSON)
}
// QueryPolicy retrieves a policy from the ledger
func (s *SmartContract) QueryPolicy(ctx contractapi.TransactionContextInterface, policyID string) (*Policy, error) {
	policyJSON, err := ctx.GetStub().GetState(policyID)
	if err != nil {
		return nil, fmt.Errorf("failed to read from ledger: %v", err)}
	if policyJSON == nil {
		return nil, fmt.Errorf("policy %s does not exist", policyID)}
	var policy Policy
	err = json.Unmarshal(policyJSON, &policy)
	if err != nil {
		return nil, err}
	return &policy, nil}
// UpdatePolicyStatus updates the status of a policy
func (s *SmartContract) UpdatePolicyStatus(ctx contractapi.TransactionContextInterface, policyID string, newStatus string) error {
	policy, err := s.QueryPolicy(ctx, policyID)
	if err != nil {
		return err}
	policy.Status = newStatus
	policyJSON, err := json.Marshal(policy)
	if err != nil {
		return err}
	return ctx.GetStub().PutState(policyID, policyJSON)}
// PolicyExists checks if a policy exists in ledger
func (s *SmartContract) PolicyExists(ctx contractapi.TransactionContextInterface, policyID string) (bool, error) {
	policyJSON, err := ctx.GetStub().GetState(policyID)
	if err != nil {
		return false, err}
	return policyJSON != nil, nil}
func main() {
	chaincode, err := contractapi.NewChaincode(&SmartContract{})
	if err != nil {
		fmt.Printf("Error creating chaincode: %v", err)
```

```
→/workspaces/fabric-samples/test-network (main) $ cd chaincode/insurance-go
→.../fabric-samples/test-network/chaincode/insurance-go (main) $ touch main.go
→.../fabric-samples/test-network/chaincode/insurance-go (main) $ go mod init insurance-go
o.mod: module insurance-go
requirements and sums:
/
→.../fabric-samples/test-network/chaincode/insurance-go (main) $ go mod tidy
 for package github.com/hyperledger/fabric-contract-api-go/contractapi
thub.com/hyperledger/fabric-contract-api-go v1.2.2
om/hyperledger/fabric-contract-api-go/contractapi in github.com/hyperledger/fabric-contract-
```

```
@Charis-Fernandes →/workspaces/fabric-samples/test-network (main) $ source scripts/envVar.sh
setGlobals 1
Using organization 1
@Charis-Fernandes →/workspaces/fabric-samples/test-network (main) $ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $ORDERER_CA \
  -C mychannel \
  -n insurance \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $PEER0_ORG1_CA \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $PEER0_ORG2_CA \
  --waitForEvent \
  -c '{"function":"CreatePolicy","Args":["P003","John","Health","5000"]}'
2026-02-25 05:06:39.262 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d4aa86a353665efa30379850ab99ad42cdbc9a01c4e4a02935d437fc6
ommitted with status (VALID) at localhost:9051
2026-02-25 05:06:39.263 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d4aa86a353665efa30379850ab99ad42cdbc9a01c4e4a02935d437fc6
ommitted with status (VALID) at localhost:7051
2026-02-25 05:06:39.263 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
@Charis-Fernandes →/workspaces/fabric-samples/test-network (main) $ peer chaincode query \
  -C mychannel \
  -n insurance \
  -c '{"Args":["QueryPolicy","P003"]}'
{"policyID":"P003","holder":"John","type":"Health","premium":5000,"status":"ACTIVE"}
@Charis-Fernandes →/workspaces/fabric-samples/test-network (main) $
```